AMRITA VISHWA VIDYAPEETHAM
SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Addressing optimization issues in game design using evolutionary algorithms

23CSE399 – Project Phase 1 (Panel Review 2)
March 30, 2026

Team Members		Guide	Project Type	Marks
CB.SC.U4CSE23152	Vinaayak Kanagaraj	**Dr Jeyakumar G** *Professor,* *Dept. of CSE*	**Type 1** Research	.../ 5
CB.SC.U4CSE23215	Darun R G			
CB.SC.U4CSE23232	Pradeep S			
CB.SC.U4CSE23235	Anusanth R			
CB.SC.U4CSE23256	Sathya Roopan M			

1 Problem Definition & Originality

- **Problem Statement:** Strategy Optimization and Co-Evolution of Competitive Game Agents Using Evolutionary Algorithms.
- **Project Scope:** Strategy and Parameter Optimization of games with co-evolving competitive game agents using evolutionary algorithms.
- **Abstract:** This work introduces a game system where agents learn and improve continuously, rather than being trained once for fixed goals. Agents are evaluated using weighted performance histories across multiple generations — capturing adaptive capability over time rather than momentary performance, while operating within a fixed reward structure that ensures consistent and reproducible fitness measurement. Instead of adapting to opponent behavior alone, agents evolve in response to shifts in the game environment itself — changes in map configuration, resource distribution, and competitive conditions across generations. A diversity-preservation mechanism prevents populations from converging to similar behaviors, while a Hall of Fame archive ensures agents are tested against historically strong opponents rather than only current ones. Together, these mechanisms aim to produce game agents that remain competitive, strategically diverse, and stable as the environment evolves — with all key components formally defined and validated on the microRTS simulation environment.

2 Literature Survey & Gap Analysis

- **Existing Solutions:** Evolutionary algorithms are widely used in game research due to their ability to explore large search spaces and generate high-quality solutions for agents, content, and game balancing. Prior work demonstrates applications in game balancing and disruption in RTS environments, dynamic difficulty adjustment and adaptive gameplay, game economy modeling and parameter tuning, procedural content generation, optimizing RTS bot strategies, and evolving cooperative and competitive agent behaviors. Co-evolution plays a key role in multi-agent systems, where agents evolve in response to each other, leading to adaptive strategies and emergent behaviors.
- **Limitations:** Most methods are tested only in simple or fixed settings, causing overfitting and poor generalization. Many rely on manual tuning, hand-crafted rules, or fixed parameter settings, reducing scalability and adaptability. Current approaches often optimize single objectives (e.g., win rate, difficulty), without considering long-term adaptation or evolving opponent behavior. They are typically trained offline and cannot adapt or model opponents in real time during execution.
- **Research Gap:** Most existing approaches are game-specific, limiting generalization across different environments. There is limited focus on time-dependent performance and strategy evolution across multiple generations. There is a need for more adaptive, trajectory-aware, and coevolutionary frameworks

that can handle dynamic and complex game environments effectively — frameworks that avoid heavy reliance on manually designed fitness functions and domain-specific assumptions.

3 Proposed Methodology (Technical Knowledge)

We propose **TD-CCEA** (Trajectory-Driven Competitive Co-Evolutionary Algorithm), a co-evolutionary framework where agents are evaluated not on a single game result, but on a weighted history of past performance — where memories from similar competitive environments count more than memories from different ones. Unlike standard evolutionary algorithms that judge agents only by their most recent game, TD-CCEA uses past performance history weighted by environment similarity to produce a stable, context-aware score for each agent. This makes the evaluation smarter, more reliable, and resistant to the noise of individual game outcomes.

Core Innovation — Trajectory Score (e_i): Each agent maintains a memory diary H_i of its last $W = 5$ generations. Every memory entry stores two things: the fitness score from that generation, and the population centroid (average DNA of all agents) at that moment. At each generation, the algorithm computes how similar each past environment was to the current one using Euclidean distance on population centroids. Memories from similar environments receive a higher weight; memories from very different environments receive a lower weight. The final trajectory score is a normalized weighted average of past fitness values:

$$\alpha_k = 1 - \frac{d(\mathbf{c}_t,\, \mathbf{c}_k)}{d_{\max}}, \qquad \hat{\alpha}_k = \frac{\alpha_k}{\sum_j \alpha_j}, \qquad e_i^t = \sum_k \hat{\alpha}_k \cdot f_i^{(k)}$$

where $\mathbf{c}_t$ is the current population centroid, $\mathbf{c}_k$ is the centroid stored in memory k, and $f_i^{(k)}$ is the fitness from that memory. A naive average would treat all past games equally. TD-CCEA instead asks: *"Was the environment back then similar to now?"* — and weights accordingly, making the score both stable and context-aware.

Self-Adapting Mutation Rate (μ_i): Instead of a fixed mutation rate applied equally to all agents, TD-CCEA adapts the mutation rate per agent based on its trajectory score:

$$\mu_i = \mu_{base} \times (1 - e_i^t) + \epsilon$$

Agents performing well (high e_i) mutate slowly — preserving good DNA. Agents performing poorly (low e_i) mutate aggressively — escaping dead ends. When $e_i = 1.0$, the mutation rate is 0.05 (fine-tuning only). When $e_i = 0.0$, the mutation rate is 0.25 (large exploratory changes). No manual tuning of mutation rates is required.

Two-Layer Diversity Preservation: Evolutionary pressure naturally drives agents toward similar strategies over time. TD-CCEA prevents this through two explicit pruning mechanisms applied each generation. First, weakness pruning removes any agent whose average trajectory score falls below $\delta_f = 0.25$ after three or more generations — eliminating consistently failing agents. Second, redundancy pruning removes agents whose DNA similarity to another agent exceeds $\delta_s = 0.85$ — eliminating near-duplicate strategies and keeping the population diverse and efficient.

Archive and Reproduction: The top-m survivors each generation are inserted into a bounded Hall of Fame archive $\mathcal{A}$ (maximum size $A_{\max}$). When the archive exceeds capacity, the agent with the lowest quality-diversity score is evicted. New offspring are bred from the combined pool of current survivors and archived agents using Gaussian mutation $\mathcal{N}(0,\, \mu_i)$, ensuring that historically strong strategies continue to contribute to future generations. The final output is the best agent $\mathcal{O}^* \in \mathcal{A}$ from the archive.

Validation Environment — microRTS: TD-CCEA is validated on microRTS, a lightweight, fully deterministic Real-Time Strategy simulator built for AI research. Each agent is represented as a 6-dimensional DNA vector — one weight per available bot strategy (WorkerRush, LightRush, HeavyRush, RandomBiased, Random, Passive). The active strategy is selected by $argmax$ of the DNA vector, keeping the search space small and directly evolvable. Training runs for 100 generations with a population of $N = 20$ agents,

each playing $K = 2$ games per generation (4000 total game simulations), executed in headless mode via `BatchGameRunner` — completing in under one minute on a single CPU with no GPU required. TD-CCEA was benchmarked against three baseline algorithms: Hill Climbing, Genetic Algorithm, and (μ, λ)-Evolution Strategy. The evolved agent achieved a **100% win rate** against all benchmark opponents across both 8×8 and 16×16 map environments.

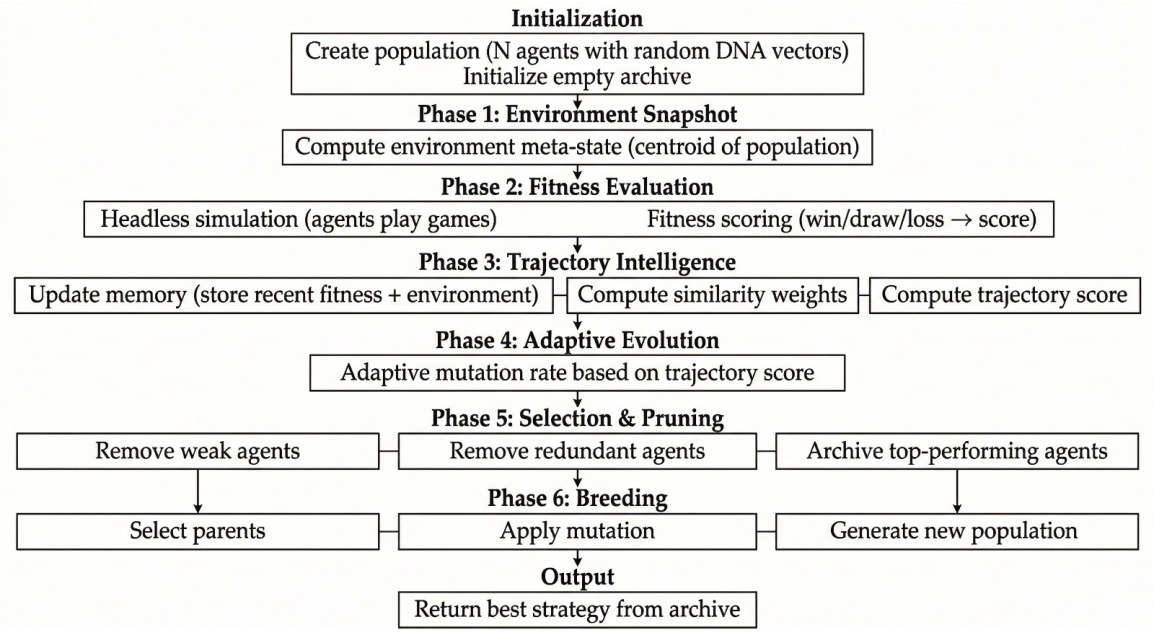


Figure 1: TD-CCEA Architecture: full iterative workflow showing environment snapshot, fitness evaluation, trajectory intelligence, adaptive evolution, selection and pruning, and breeding phases.

4 Current Status & Plan

Status: Literature survey completed; problem formulation done; TD-CCEA algorithm designed and validated on microRTS with 100% win rate against benchmark opponents on 8×8 and 16×16 maps. Reviewer comments from the first review have been addressed: baseline comparisons (GA, ES, Hill Climbing) identified; parameter space reduced to a compact 6-dimensional DNA vector; references updated to include works from 2018–2025.

Plan: Extend the framework in two phases as described below.

5 Future Work

Semester 2 — Algorithm Extensions:

- **Multi-player scaling:** Extend TD-CCEA to support 10–100 simultaneous competing agents, analysing emergent coalition and arms-race dynamics at scale.
- **Probabilistic search reduction:** Integrate methodologies such as intentional state-violation coupled with backpropagation, inter-sequence compatibility, and Markov Models to constrain the evolutionary search space. This is motivated by DipLLM [4], which required only 1.5% of the training space of Meta's Cicero model. The approach will be combined with mutated exploration and backpropagation of trial search paths.
- **Decentralized fitness computation:** Distribute competition and fitness evaluation across spatial maps using concurrent execution, reducing per-generation wall-clock time for large populations.

Semester 3 — Framework Generalization:

- **Abstracted strategic games:** Adapt the TD-CCEA framework to Diplomacy (binary alliance–betrayal model, no inter-agent messaging), Battleship, and Checkers — testing generalization across increasingly complex decision spaces.
- **Biological evolution modeling:** Simulate a population evolving through diversified sequential catastrophic environmental shifts, each causing partial collapse and rapid changes in fitness landscapes, to

study how evolutionary mechanisms respond to drastic regime changes.

References

[1] J. Snell, M. Masek, and C. P. Lam, "An evolutionary approach to balancing and disrupting real-time strategy games," in *Proceedings of the 24th International Congress on Modelling and Simulation (MODSIM2021)*, Sydney, NSW, Australia, December 2021, pp. 967–973.

[2] L. T. Pereira, B. M. F. Viana, and C. F. M. Toledo, "Procedural enemy generation through parallel evolutionary algorithm," in *2021 20th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, 2021, pp. 126–133.

[3] D. Novak, D. Verber, J. Dugonik, and I. J. Fister, "A comparison of evolutionary and tree-based approaches for game feature validation in real-time strategy games with a novel metric," *Evolutionary Computation 2020*, 2020.

[4] K. Xu, J. Chai, S. Li, Y. Fu, Y. Zhu, and D. Zhao, "DipLLM: Fine-tuning LLM for strategic decision-making in diplomacy," in *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[5] T. Anthony, T. Eccles, A. Tacchetti, J. Kramár, I. Gemp, T. C. Hudson, N. Porcel, M. Lanctot, J. Pérolat, R. Everett, R. Werpachowski, S. Singh, T. Graepel, and Y. Bachrach, "Learning to play no-press diplomacy with best response policy iteration," in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020.

[6] Y. Widhiyasana, M. Harika, F. F. N. Hakim, F. Diani, K. S. Komariah, and D. R. Ramdania, "Genetic algorithm for artificial neural networks in real-time strategy games," *International Journal on Informatics Visualization*, 2022.

[7] A. Rawal, P. Rajagopalan, and R. Miikkulainen, "Constructing competitive and cooperative agent behavior using coevolution," in *Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG)*, IEEE, 2012, pp. 1–8.

[8] G. L. Zuin, Y. P. A. Macedo, L. Chaimowicz, and G. L. Pappa, "Discovering combos in fighting games with evolutionary algorithms," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2016)*, Denver, CO, USA: ACM, 2016, pp. 277–284.

[9] S. M. Lucas, J. Liu, I. Bravi, R. D. Gaina, J. Woodward, V. Volz, and D. Perez-Liebana, "Efficient evolutionary methods for game agent optimisation: Model-based is best," in *Proceedings of the AAAI Workshop on Games and Simulations for Artificial Intelligence*, AAAI Press, 2019, pp. 1–8.

[10] H. Armanto, H. A. Rosyid, Muladi, and Gunawan, "Evolutionary algorithm in game – a systematic review," *Kinetik: Game Technology, Information System, Computer Network, Computing, Electronics, and Control*, vol. 8, no. 2, pp. 599–606, 2023.

[11] K. O. Andrade, R. C. Joaquim, G. A. P. Caurin, and M. K. Crocomo, "Evolutionary algorithms for a better gaming experience in rehabilitation robotics," *ACM Computers in Entertainment*, vol. 16, no. 2, pp. 1–15, 2018.

[12] M. A. Fajardo, P. K. Lehre, and S. Lin, "Runtime analysis of a co-evolutionary algorithm: Overcoming negative drift in maximin-optimisation," in *FOGA '23: Foundations of Genetic Algorithms XVII*, 2023.

[13] D. Perez-Liebana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game AI: A multitrack framework for evaluating agents, games, and content generation algorithms," *IEEE Transactions on Games*, vol. 11, no. 3, pp. 195–214, 2019.

[14] A. Fernandez-Ares, A. M. Mora, J. J. Merelo, P. Garcia-Sanchez, and C. Fernandes, "Optimizing player behavior in a real-time strategy game using evolutionary algorithms," in *Proceedings of the 2011 IEEE Congress on Evolutionary Computation (CEC)*, New Orleans, USA: IEEE, 2011, pp. 2017–2024.

[15] R. D. Gaina, S. Devlin, S. M. Lucas, and D. Perez-Liebana, "Rolling horizon evolutionary algorithms for general video game playing," *IEEE Transactions on Games*, vol. 14, no. 2, pp. 232–242, 2022.

[16] R. Guha, R. Mckendrick, B. Feest, and K. Deb, "Multi-objective competitive co-evolutionary optimization and regularity-based decision-making for two-agent wargame strategy optimization," *IEEE Transactions on Evolutionary Computation*, 2025.